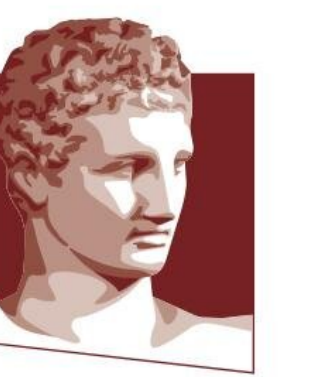


Evaluation and Development of New Cache Replacement Policies: The Histogram-based LRU Policy

Eleni Driva¹, Vana Kalogeraki¹

¹Department of Informatics, Athens University of Economics and Business, Greece



ΟΠΑ
AUEB

1. Introduction

- ▶ Modern systems require scalability, high availability, and high throughput to meet performance demands.
- ▶ Memory caches like Redis play a critical role, yet have limited capacity, requiring smart replacement decisions.
- ▶ Traditional policies like LRU often struggle under mixed access patterns in systems like CDNs, edge computing.
- ▶ There is a need for adaptive replacement policies that consider temporal locality and access frequency.

2. Replacement Policies

LRU (Least Recently Used) Replacement Policy:

- ▶ Exploits temporal locality - favors recently used items.
- ▶ Struggles with bursty/periodic patterns and mixed workloads - evicts useful data.
- ▶ Pollutes the cache during sequential scan.
- ▶ **Approximated LRU:**
 - ▶ Samples a configurable number of candidates (*maxmemory-samples*) and evicts the least recently used among them.
 - ▶ Closely approximates the theoretical LRU (5 samples).
 - ▶ Low computational and memory overhead.

Other Policies:

- ▶ **LFU (Least Frequently Used):**
 - ▶ Tracks access frequency - favors consistently accessed data.
- ▶ **Random:**
 - ▶ Replaces items at random - low complexity, low hit rate.

3. Motivation

- ▶ **Objective:**
Develop a new cache replacement policy on top of the **Redis state-of-the-art approximated LRU**, improving the evaluation of data usefulness through temporal locality and access frequency, resulting in **improved application performance**.
- ▶ **Challenges:**
 - ▶ Memory overhead: Use the minimum extra memory resources
 - ▶ Fine-tuning: Efficiency in both simple and complex workloads

4. Our Solution: Histogram-based LRU

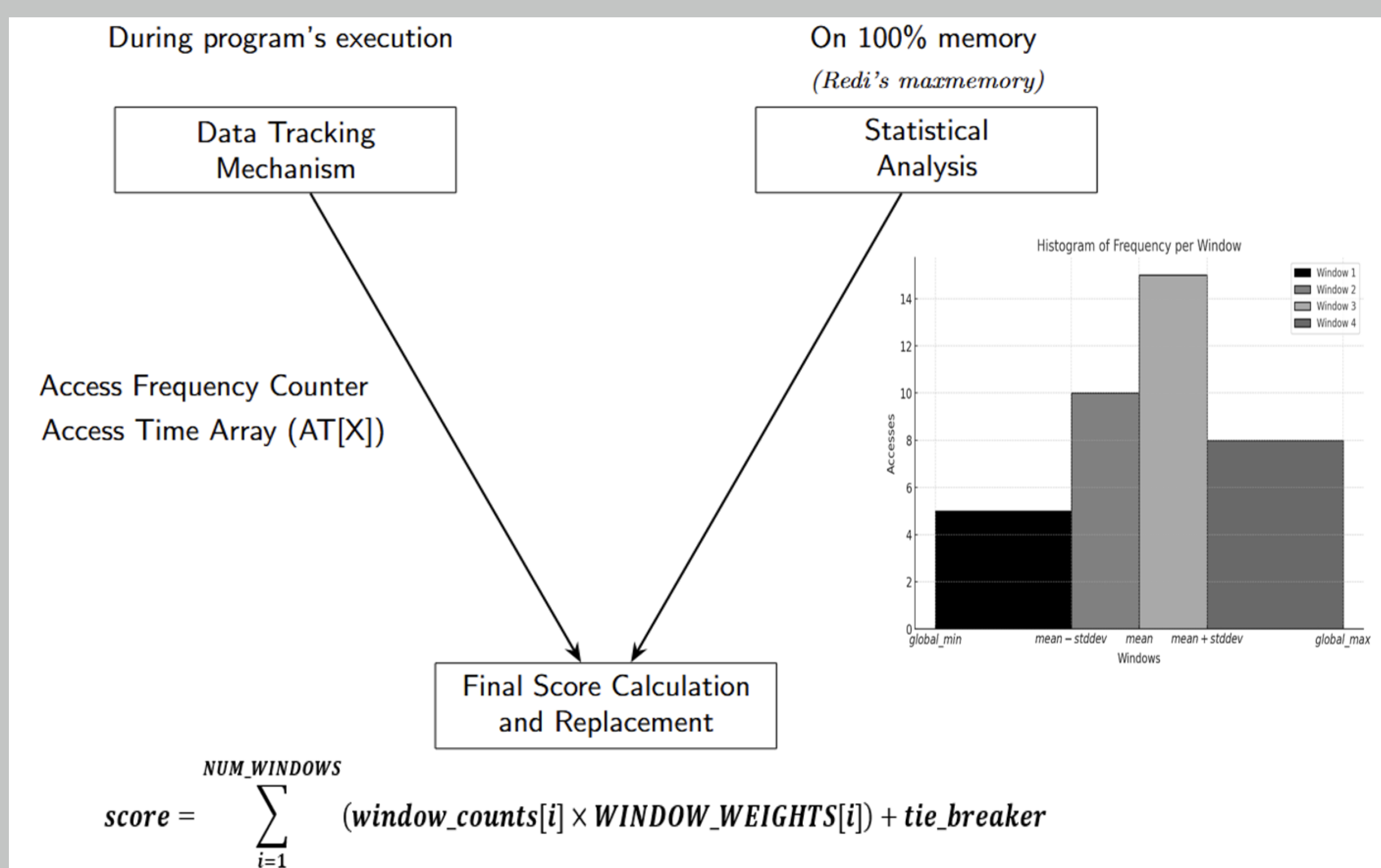


Figure: Histogram – based LRU - Overview

- ▶ **Histogram – based LRU:** Scoring-based cache replacement using access frequency and temporal distribution.
- ▶ Data Tracking Mechanism
- ▶ Statistical Analysis
- ▶ Final Score Calculation

5. HLRU Components

▶ Data Tracking Mechanism

- ▶ **Frequency Counter:** Tracks total access count (8 bits).
- ▶ **Access Timestamp Array (AT[X])** - Configurable size X: Circular buffer storing the last X access times (24-bit relative timestamps), balancing precision and memory efficiency.

▶ Statistical Analysis

- ▶ **Access ages:** time elapsed since access ($age = now - AT$).
- ▶ **Global statistics:** Computes *mean*, *standard deviation*, *min*, and *max* of ages for sampled candidates.
- ▶ **Dynamic time windows:** Created using statistical bounds from computed metrics to group accesses by recency - adaptive to both sparse and dense patterns.

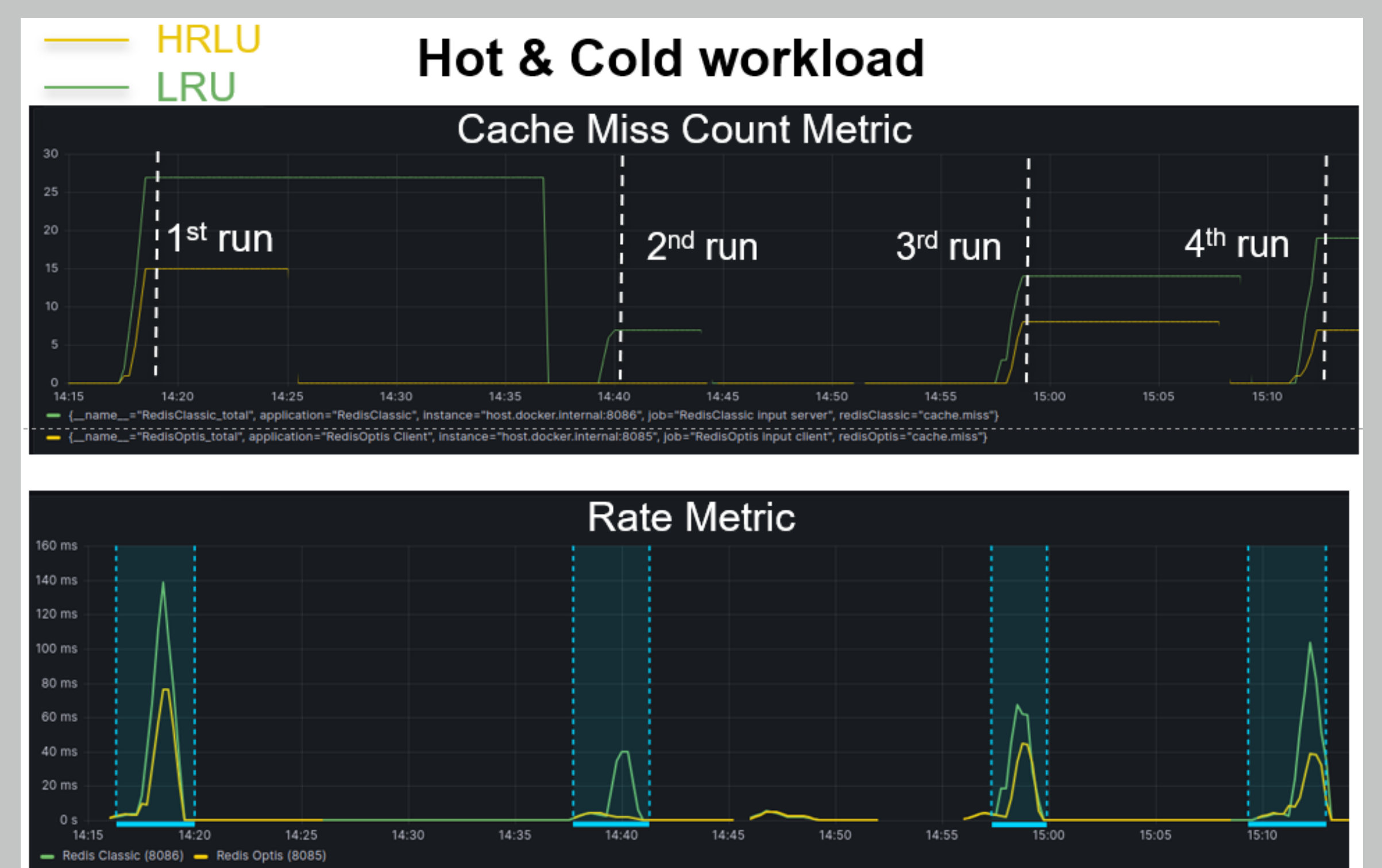
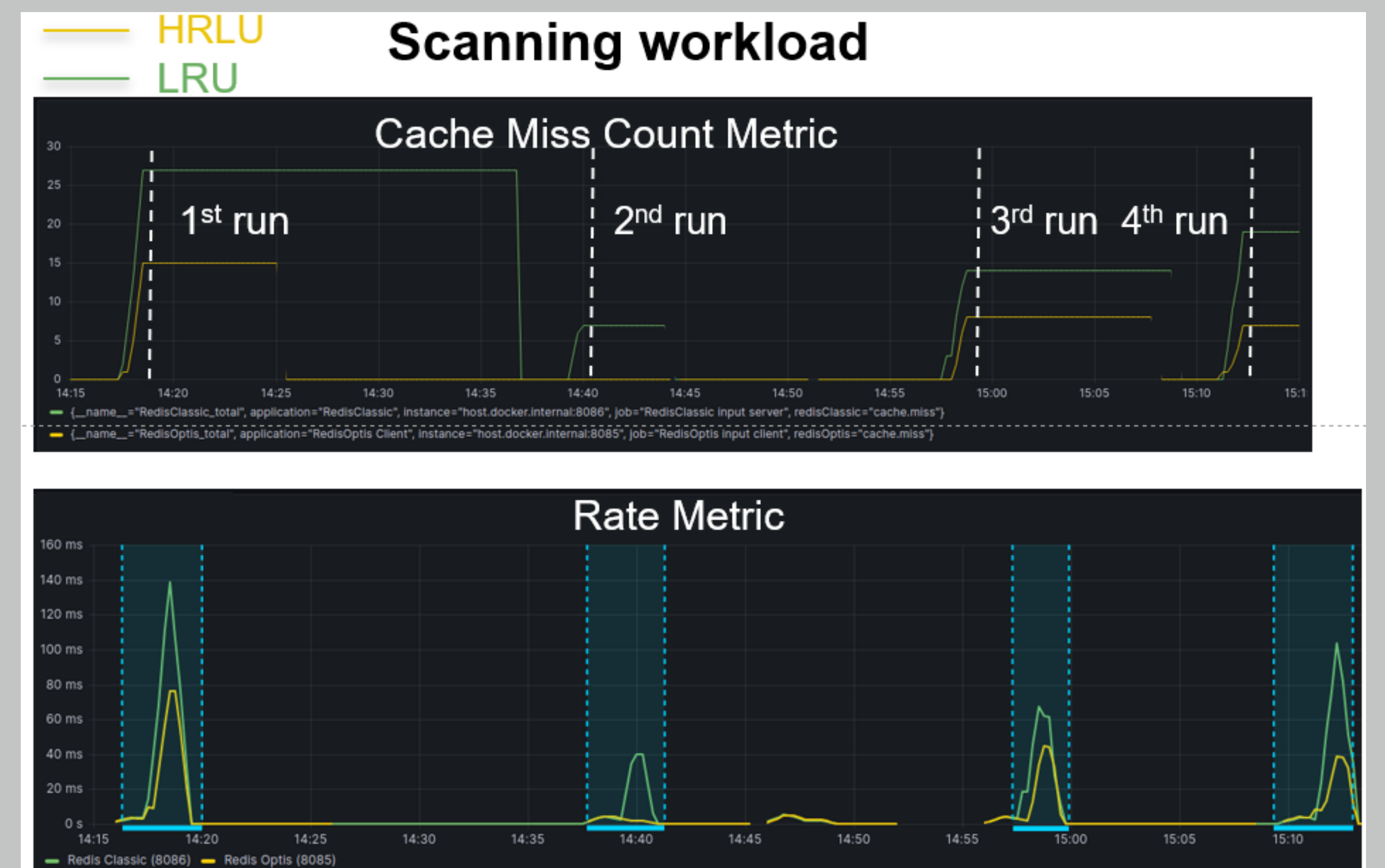
▶ Scoring - Histogram Creation

- ▶ **Window Counts:** Number of accesses per time window.
- ▶ **Window Weights:** Higher for recent windows, reflecting higher utility.
- ▶ **Tie-breaker:** Small adjustment based on total access frequency.

6. Evaluation

Scenarios

- ▶ **Random / Sequential**
 - ▶ **Recency-driven**
 - ▶ **Hot & Cold**
 - ▶ **Scanning**
 - ▶ **HLRU AT = {1, ..., 5}**
- ### Evaluation
- ▶ **Random/Sequential:** HLRU = LRU
 - ▶ **Recency-driven:** LRU slightly better
 - ▶ **Hot & Cold:** -52.3% cache misses vs LRU
 - ▶ **Scanning:** -45.1% cache misses vs LRU
 - ▶ **Slight memory increase** for increasing AT



Contact Information

- ▶ Eleni Driva
 - ▶ Email: edrive@aub.gr
- ▶ Vana Kalogeraki
 - ▶ Email: vana@aub.gr
 - ▶ Web: <http://www2.cs.aub.gr/~vana/>
- ▶ Real-Time Distributed Systems Group
 - ▶ Web: <http://rtids.aub.gr>